New Energy Fleet Transport System

Experiment Report

Course: DataStructure

Date: May 28, 2026

Member 1: Ruixi Liang
Member 2: Junxi Chen
Member 3: Zichao Yang
Member 4: Jinyi Pang

Abstract

This experiment addresses the cooperative dispatching problem for electric logistics fleets in urban environments by designing and implementing a complete simulation system. The system uses a graph-based road network constructed via OSMnx and Dijkstra's shortest path algorithm. The fleet consists of three types of electric vehicles (small, medium, large), each with battery capacity and payload constraints. During simulation, delivery tasks are dynamically generated; all timestamps and deadlines use simulation time. A unified scoring mechanism evaluates task completion consistently across strategies.

Ten dynamic dispatching strategies are implemented and compared: four greedy baselines, three search-based batch loaders, and three multi-vehicle coordination algorithms. Benchmarks use shared task-generation seeds across three scales (small: 3 vehicles; medium: 6 vehicles; large: 12 vehicles).

Dynamic results show near-perfect completion rates with clear score separation: search-based methods (DFS Score Search, RL Batch) dominate at medium scale (best score 3824.3), while Cluster Auction MAS leads at large scale (3278.7). A supplementary Gurobi static solver baseline does not consistently outperform dynamic policies; several dynamic strategies score higher at small and medium scales, possibly due to time-penalty modeling mismatch in the MIP formulation. The system also implements charging-station queuing, battery feasibility checks, and an AMap-based visualization interface.

Keywords: Electric logistics, vehicle routing problem, cooperative dispatching, graph search, metaheuristic algorithm

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Background

Driven by carbon neutrality policies and the development of urban green logistics, new energy logistics vehicles have been widely adopted in urban distribution. Unlike traditional fuel vehicles, electric logistics vehicles are restricted by battery capacity and payload limits, while also facing range anxiety, necessary recharging demands and queuing pressure at charging stations. Urban delivery tasks arrive dynamically with random locations and cargo weights, making manual scheduling inefficient in balancing timeliness, travel cost and energy consumption.

This project establishes a simulation system from the perspective of a central warehouse manager to realize cooperative scheduling and route planning of new energy fleets. Dispatching algorithms for such fleets are typically complex and difficult to compare in isolation; we therefore build a unified simulation framework so that multiple strategies can be evaluated under identical conditions. Under the constraints of road network, battery power, load capacity and charging station occupancy, multiple dispatching strategies are designed and compared to provide practical references for urban logistics operation.

1.2 Problem Definition

Problem Statement: Given a city delivery environment consisting of a central warehouse, task locations, and charging stations, a fleet of electric vehicles must complete dynamically arriving delivery tasks under the following constraints:

- **Fleet constraints:** Limited number of vehicles, each with maximum battery capacity $B_{\max}$ and maximum payload $W_{\max}$, varying by vehicle type.
- **Task constraints:** Each task has a creation time t_{create}, location coordinates (x, y), cargo weight w, deadline t_{deadline}, and priority p.
- **Charging constraints:** Vehicles can recharge at charging stations, which have limited charging slots and queuing delays.
- **Objective:** Maximize the total reward (score), where tasks completed earlier with shorter travel distances yield higher rewards, while overdue tasks incur penalties.

1.3 Experiment Objectives

The main objectives of this experiment are:

1. Design and implement a complete urban logistics simulation system, covering road network modeling, vehicle modeling, task generation, charging station modeling, and scoring mechanisms.
2. Implement multiple electric vehicle dispatching strategies, including greedy baselines, search-based batch loaders, and multi-vehicle coordination algorithms — ten strategies in total.
3. Conduct systematic comparative tests across three problem scales (small, medium, large), with shared task-generation seeds to ensure fair comparison.
4. Analyze the suitability of each strategy under different scales and determine which strategy performs best under which conditions.
5. Design an AMap-based graphical interface to visualize the vehicle dispatching process, real-time status, and statistical data.

1.4 Report Structure

The report is organized as follows:

- **Chapter 2 Simulation Design:** Details the overall system architecture, simulation-time model, unified scheduling interface, and core data models.
- **Chapter 3 Algorithm Design:** Describes the design philosophy and implementation of all ten dispatching strategies.
- **Chapter 4 Experiment Design:** Explains the experimental environment, three-scale configuration, experimental methodology, and evaluation metrics.
- **Chapter 5 Results and Analysis:** Presents experimental results for each scale and algorithm, with comparative analysis using tables and charts.
- **Chapter 6 Conclusion:** Summarizes the project outcomes, team member contributions, and lessons learned.

Chapter 2

Simulation Design

2.1 System Architecture

The system adopts a B/S architecture with separated front-end and back-end. The back-end implements the simulation core logic in Python, while the front-end provides visualization and interaction via HTML/JavaScript. Figure 2.1 illustrates the overall system architecture.

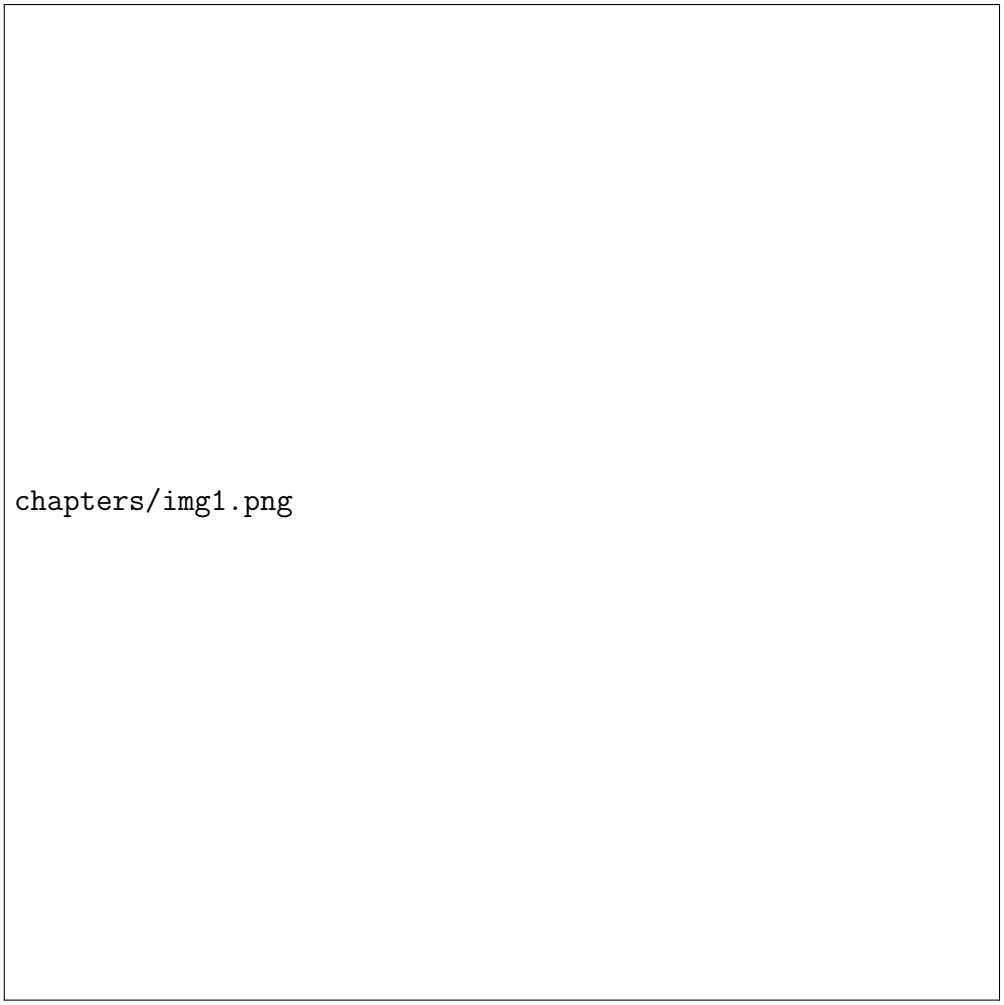


Figure 2.1: System Architecture

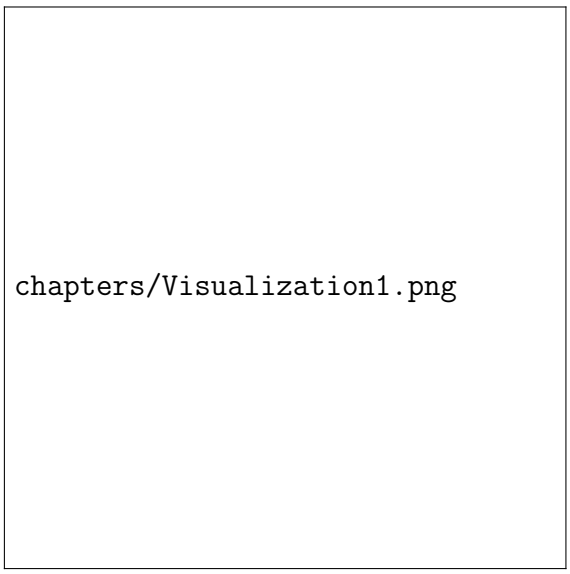
The responsibilities of each layer are as follows:

- **Visualization Layer:** Displays vehicle real-time positions, task locations, and charging station status on AMap; uses Chart.js to plot completion rates and score trends; receives real-time data pushed via WebSocket.

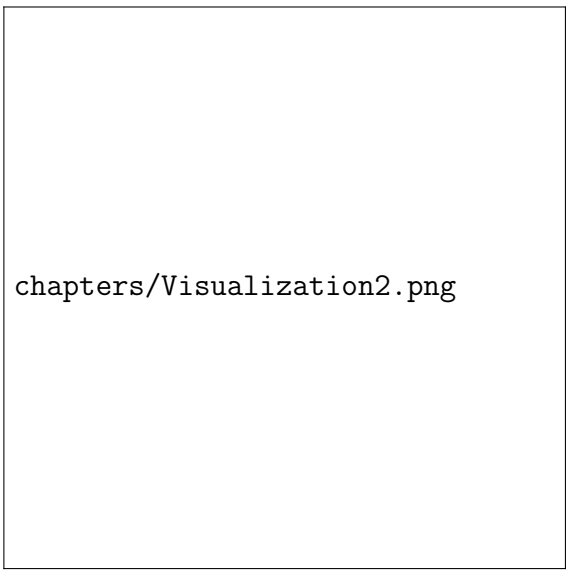
- **API Layer:** Builds RESTful and WebSocket interfaces based on FastAPI, handling simulation control (start, pause, reset).
- **Decision Layer:** Coordinates the simulation main loop, captures a system snapshot at each scheduling tick, invokes the algorithm layer to produce command lists, and executes them on the data layer.
- **Algorithm Layer:** Registers all dispatching strategies through a central algorithm manager; each strategy reads the snapshot and returns per-vehicle next-step commands.
- **Data Management Layer:** Maintains the state of all entities (vehicles, tasks, charging stations) and exposes a snapshot mechanism for read-only algorithm access.
- **Road Network:** Builds city road topology from OpenStreetMap via OSMnx, providing Dijkstra shortest-path queries.

2.1.1 Visualization Highlights

The visualization interface brings the system architecture to life, providing real-time insight into the dispatching process as it unfolds on the road network. Figure 2.2 showcases the AMap-based dashboard.



(a) Real-time vehicle positions and task distribution on AMap



(b) Charging station status, route paths, and fleet coordination

Figure 2.2: System Visualization Dashboard — Real-time Fleet Monitoring Interface

As shown in Figure 2.2, the interface offers several key capabilities:

- **Real-time vehicle tracking:** Each vehicle's position updates every simulation tick via WebSocket push, with distinct markers for different vehicle types (small/medium/large). Vehicle status (IDLE, MOVING, CHARGING) is color-coded for quick identification.

- **Task distribution overview:** Pending, in-progress, and completed tasks are displayed with different markers on the map, providing an intuitive understanding of workload distribution across the road network.
- **Charging station monitoring:** Station occupancy, queue length, and load pressure are displayed in real time, helping to verify that the charging decision algorithm correctly balances load across stations.
- **Route visualization:** Calculated delivery routes are drawn as polylines on the map, allowing visual inspection of route quality — crossing paths, unnecessarily long detours, and charging station detours are immediately visible.
- **Statistical dashboard:** Alongside the map, Chart.js renders live-updating charts for completion rate, cumulative score, and fleet utilization trends, enabling at-a-glance performance assessment during simulation.

This visualization proved invaluable during development. For example, watching a vehicle take an illogical detour immediately revealed a bug in the charging station selection logic that would have been difficult to catch from log files alone. Similarly, the route visualization made it easy to compare the route quality of different algorithms side by side.

2.2 Unified Scheduling Interface

Dispatching algorithms for electric logistics fleets must simultaneously handle task selection, batch loading, route ordering, battery feasibility, and charging-station choice. Implementing each strategy as a standalone script makes fair comparison difficult: results are only meaningful when every algorithm runs under **identical simulation conditions**—the same road network, fleet parameters, task generation sequence, and scoring rules.

To address this, the simulation core exposes a **snapshot-driven, unified scheduling interface**:

- **Snapshot as input:** Before each decision round, the system captures a read-only snapshot of the current world state—vehicle positions, battery levels, pending tasks, charging-station occupancy, and the current simulation time t_{sim}. Algorithms never mutate state directly; they only read the snapshot and return commands.
- **Command as output:** Each algorithm produces a list of commands, one per vehicle that needs a decision, specifying the *next* action (deliver, charge, return, handoff, etc.). Multi-stop delivery emerges from repeated snapshot-to-decision cycles rather than from a monolithic plan.
- **Shared utility toolbox:** A common utility module provides default helpers for distance queries, battery pre-checks with mid-route recharge simulation, greedy route ordering, charging-station scoring, and standard decision templates for warehouse and en-route contexts. Developers can compose these defaults or override only the task-selection logic to implement custom strategies with minimal boilerplate.

- **Pluggable registration:** New schedulers inherit a common base class, declare a strategy name, and register with the algorithm manager. All strategies are selectable through a single configuration field, enabling head-to-head benchmarks.

This design ensures that greedy heuristics, search-based batch loaders, multi-agent auction schemes, and relay-coordination strategies all operate on the same physical model and are evaluated with the same metrics. Chapter 3 describes how individual algorithms plug into this interface.

2.3 Simulation Time Model

A central design choice is to decouple **simulation time** from **wall-clock time**. All task timestamps, deadlines, timeouts, and scoring calculations use simulation seconds (t_{sim}), ensuring that algorithm comparisons are independent of hardware speed or UI rendering overhead.

2.3.1 Clock Advancement

The simulation advances in discrete ticks. Each tick:

- Waits one tick interval in real seconds (default 1.0 s).
- Advances simulation time by Δt_{sim} = speed factor $\times$ tick interval (default speed factor = 120, so 1 real second $\approx$ 120 simulation seconds).
- Moves vehicles along their routes, updates battery levels, and charges vehicles at stations proportionally to Δt_{sim}.

The snapshot timestamp and all task creation/deadline values refer to this simulation clock. The experiment log records both elapsed simulation time and elapsed wall-clock time for transparency.

2.3.2 Stop Conditions

The simulation terminates when:

- All tasks reach a terminal state (completed or timed out), and the auto-stop-on-completion option is enabled.
- The user manually stops the simulation.

Importantly, the maximum simulation seconds setting is **not** a hard simulation stop. It marks the cutoff for *new task generation*: once t_{sim} reaches this threshold (or the total task budget is exhausted), no further tasks are injected, but the simulation continues until all existing tasks are settled. This prevents premature termination while still bounding task volume.

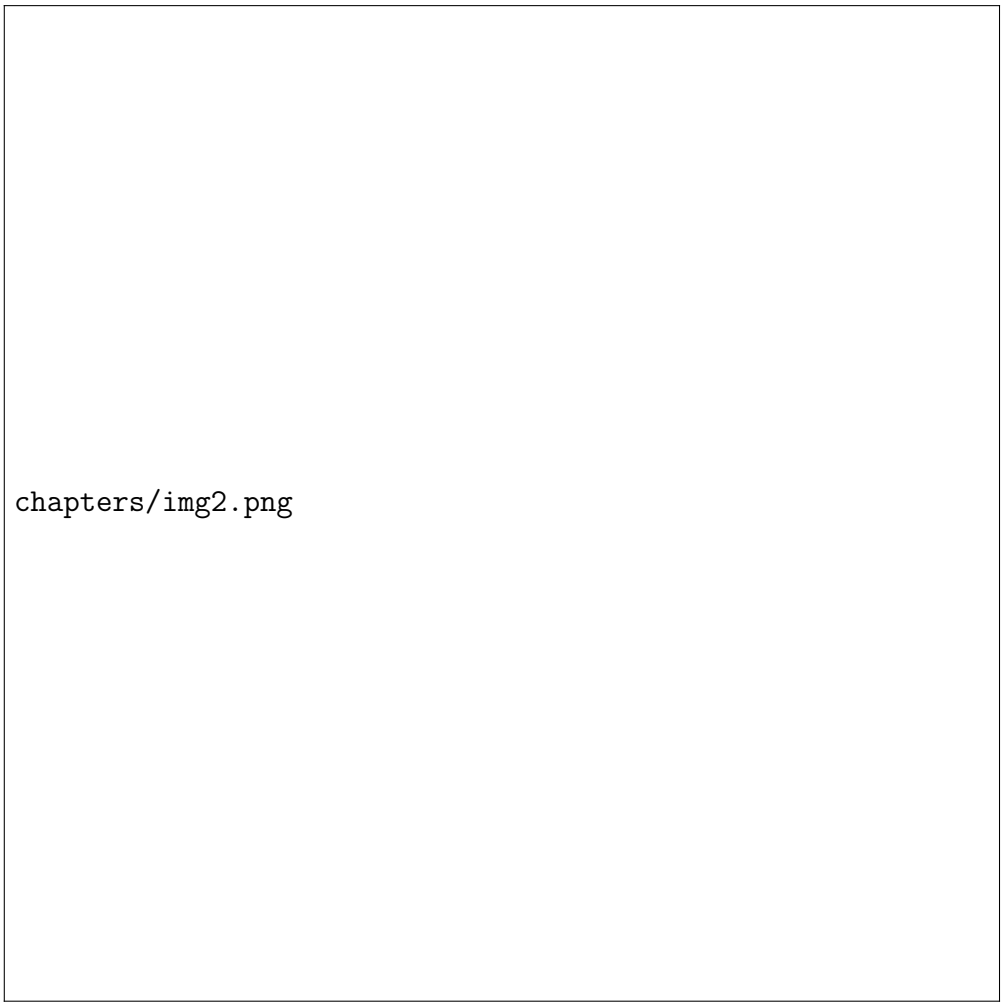


Figure 2.3: Simulation Main Loop

2.4 Simulation Main Loop

The simulation core is a time-step-driven main loop, as illustrated in Figure 2.3. Each tick consists of four core steps:

1. **Physics Update:** Advance all MOVING vehicles along their Dijkstra routes by Δt_{sim}; update battery consumption; charge vehicles at stations; handle task pickup/delivery and charging queuing.
2. **Clock & Task Generation:** Increment t_{sim}; generate new random tasks if within budget and before the generation cutoff; mark overdue tasks as timed out.
3. **Dispatch Decision:** Capture a snapshot of the current world state, invoke the selected scheduling strategy, and execute the returned command list (assign routes, load tasks, trigger handoffs).
4. **Broadcast:** Push updated vehicle positions, scores, and statistics to the frontend via WebSocket.

The dispatch step follows a strict read-only contract: algorithms observe the snapshot and emit commands; all state mutations happen in the dynamic scheduling execution layer.

2.5 Road Network Modeling

2.5.1 Graph Definition

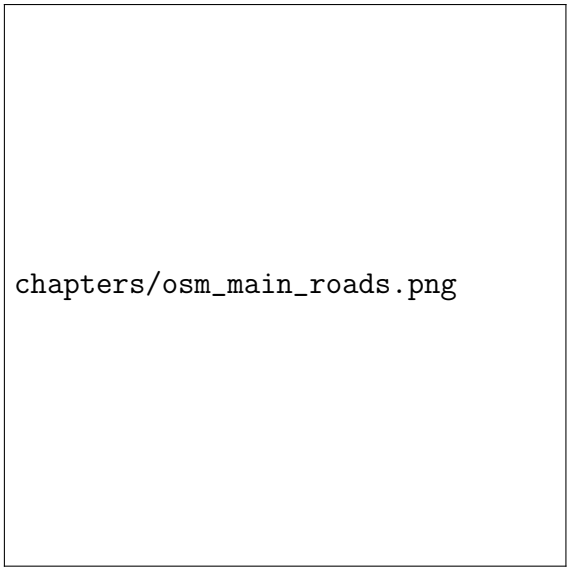
The road network is modeled as an undirected weighted graph $G = (V, E, w)$, where:

- V is the set of road intersections (nodes).
- $E \subseteq V \times V$ is the set of road segments (edges).
- $w : E \to \mathbb{R}^+$ is the edge weight function representing road segment length in meters.

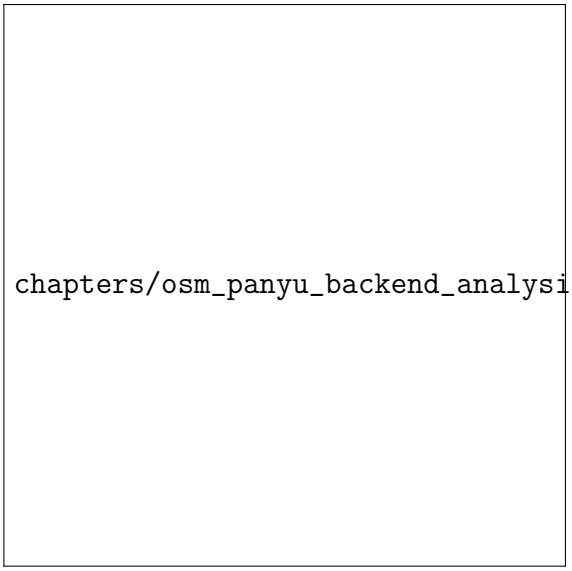
The system builds the graph from OpenStreetMap data (Panyu District, Guangzhou) via OSMnx, with optional main-road filtering to reduce graph size while preserving major arterials.

2.5.2 Shortest Path Algorithm

Shortest paths between two points are computed using Dijkstra's algorithm. For given origin s and destination t, the algorithm returns the shortest distance $d(s, t)$ and the corresponding sequence of path points. The snapshot layer implements a distance cache mechanism, computing each origin-destination pair only once, significantly improving query efficiency during dispatching.



(a) Panyu District Main Road Network



(b) Road Network with Traffic Analysis

Figure 2.4: Real Road Network Based on OpenStreetMap (Panyu District)

2.6 Vehicle Model

Vehicles are the basic execution units of the dispatching system. Three vehicle types are defined with parameters shown in Table **??**.

Table 2.1: Vehicle Type Parameters

Type	**Battery** (kWh)	**Payload** (kg)	**Energy** (kWh/m)	**Charge Power** (kWh/s)	**Speed** (m/s)
Small	60	500	1.0×10^{-3}	0.050	10.0
Medium	100	1500	1.5×10^{-3}	0.100	10.0
Large	150	5000	2.0×10^{-3}	0.200	10.0

The vehicle state machine is shown in Figure **??**.

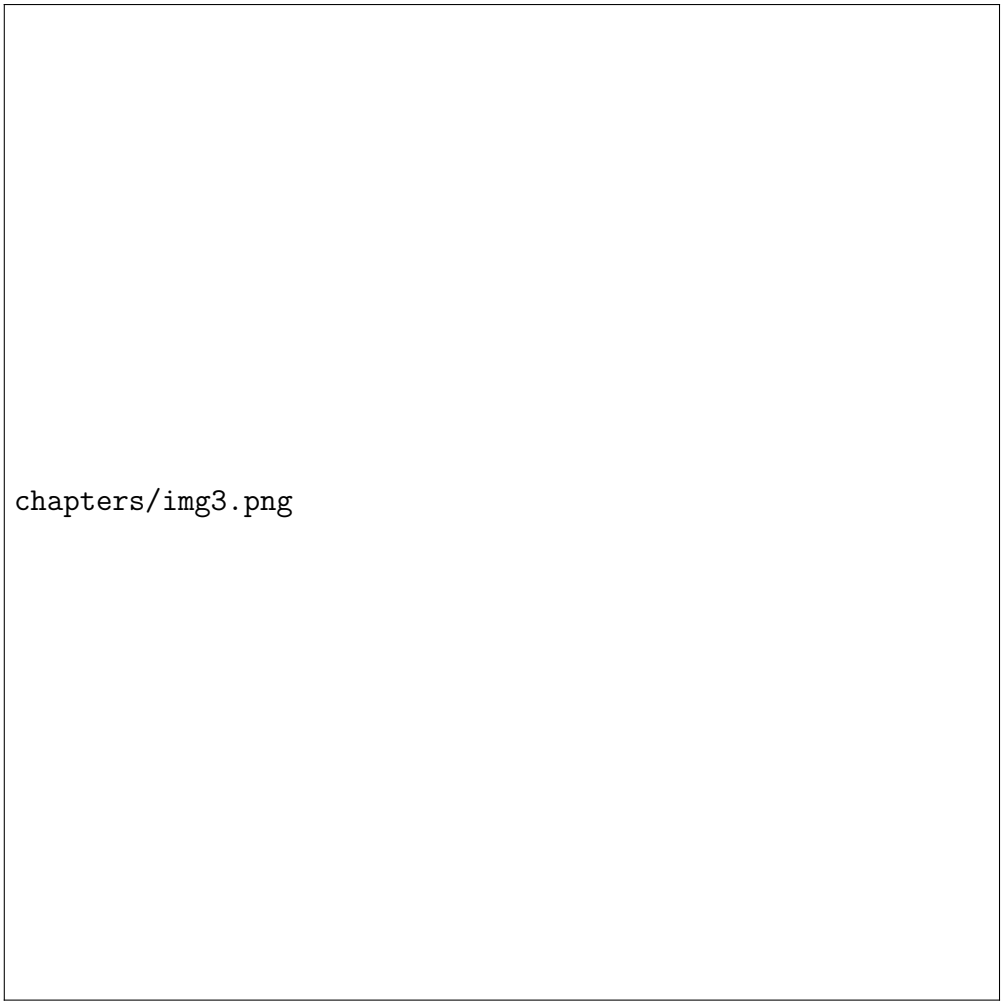


Figure 2.5: Vehicle State Machine

The energy consumption follows a linear model:

$$E_{\text{consumed}} = d \times \eta \tag{2.1}$$

where d is travel distance (meters) and η is the vehicle's unit energy consumption (kWh/m).

2.7 Task Model

2.7.1 Task Definition

Each task T is defined as a 6-tuple:

$$T = (t_{\text{create}}, x, y, w, t_{\text{deadline}}, p) \tag{2.2}$$

The fields are:

- t_{create}: Task creation time (**simulation timestamp** in seconds).
- (x, y): Delivery destination coordinates (road network node).

- w: Cargo weight (kg), must not exceed vehicle remaining payload.
- t_{deadline}: Deadline (simulation seconds); overdue tasks incur penalties.
- p: Priority (1–5), higher values indicate greater importance.

A task becomes available for assignment only when $t_{\text{sim}} \geq t_{\text{create}}$.

2.7.2 Task Generation

Tasks are generated randomly during simulation according to configurable parameters:

- **Generation interval:** Uniform distribution between minimum and maximum values in simulation seconds.
- **Batch size:** Uniform distribution between configured minimum and maximum batch sizes.
- **Total budget:** Stops generating after reaching the configured total task budget.
- **Generation cutoff:** Stops generating once t_{sim} reaches the maximum simulation seconds threshold.
- **Location:** Randomly selected from road network nodes.
- **Maximum pending tasks:** Limits the unassigned task queue size.
- **Reproducibility:** A saved task-generation seed file can be reused so that all algorithms in a benchmark share the identical task sequence.

2.7.3 Task State Machine

A task progresses through the following states during its lifecycle:

$$\text{Pending} \rightarrow \text{Assigned} \rightarrow \text{In Progress} \rightarrow \text{Completed} \mid \text{Timed Out} \tag{2.3}$$

- **Pending:** Awaiting assignment.
- **Assigned:** Assigned to a vehicle but delivery not yet started.
- **In Progress:** Vehicle is en route to the task location.
- **Completed:** Successfully delivered, score awarded.
- **Timed Out:** Deadline passed without completion.

2.8 Charging Station Model

Charging stations are a critical component of electric logistics systems. The model incorporates the following elements:

2.8.1 Basic Attributes

Each charging station S is defined as:

$$S = (\text{id}, x, y, \text{capacity}, N_{\text{queue}}, \text{load pressure}) \tag{2.4}$$

Capacity is the number of charging slots (maximum vehicles charging simultaneously), N_{queue} is the current total occupancy (charging + waiting), and load pressure is the comprehensive load indicator.

2.8.2 Queuing Mechanism

When a vehicle arrives at a charging station, the system decides based on current occupancy:

- If current charging vehicles < capacity, enter charging immediately.
- If all slots are occupied, join the waiting queue.
- When a vehicle finishes charging and departs, the first vehicle in the waiting queue is automatically promoted.

2.8.3 Load Pressure Evaluation

To quantify the busyness of a charging station, we define the load pressure metric:

$$\text{load pressure} = \min\left(1.0, \frac{N_{\text{charging}} + N_{\text{waiting}}}{\text{capacity} \times 2}\right) \tag{2.5}$$

This metric is used by dispatching algorithms when selecting a charging station, combining distance and load pressure to find the optimal choice:

$$\text{cost}(s) = d(v, s) + \text{load pressure}(s) \times w_{\text{load}} + N_{\text{waiting}} \times w_{\text{queue}} \tag{2.6}$$

where w_{load} and w_{queue} are weight coefficients (defaults: 8000 m and 2000 m respectively).

2.9 Scoring Mechanism

A comprehensive scoring mechanism is designed to quantitatively evaluate the performance of each dispatching strategy. The same formula is used for final task settlement and for batch-selection objective functions inside search-based algorithms.

2.9.1 Task Score

The score for a single task T at assignment time is:

$$\text{score}(T) = R_{\text{base}} + p \times R_{\text{priority}} - d_{\text{round}} \times P_{\text{dist}} + R_{\text{early}} - P_{\text{overdue}} \tag{2.7}$$

where $R_{\text{early}} = \max(0, (t_{\text{deadline}} - t_{\text{finish}})/60) \times R_{\text{early/min}}$ and $P_{\text{overdue}} = \max(0, (t_{\text{finish}} - t_{\text{deadline}})/60) \times P_{\text{overdue/min}}$. All times are in simulation seconds.

Parameters are listed in Table **??**.

Table 2.2: Scoring Parameters

Parameter	Description	Value
R_{base}	Base assignment reward	120.0
R_{priority}	Per-priority-level reward	30.0
P_{dist}	Distance penalty coefficient	0.02 / m
$R_{\text{early/min}}$	Early completion reward / min	2.0
$P_{\text{overdue/min}}$	Overdue penalty / min	50.0

2.9.2 Overall Evaluation Metrics

In the experimental analysis, algorithm performance is evaluated using the following metrics:

1. **Completion Rate:** $\frac{N_{\text{completed}}}{N_{\text{total}}} \times 100\%$, reflecting task processing capability.
2. **On-time Rate:** $\frac{N_{\text{on-time}}}{N_{\text{completed}}} \times 100\%$, reflecting timeliness.
3. **Total Score:** $\sum \text{score}(T)$, reflecting overall reward.
4. **Average Completion Time:** Average simulation time from task creation to completion, reflecting response speed.
5. **Throughput:** Tasks completed per simulation hour, reflecting system efficiency.
6. **Total Fleet Distance:** Total distance traveled by all vehicles, reflecting energy cost.
7. **Distance StdDev:** Standard deviation of distance traveled per vehicle, reflecting load balance.
8. **Simulation Duration:** Elapsed simulation time at termination, distinct from wall-clock runtime.

Chapter 3

Algorithm Design

This chapter details the design philosophy and implementation of each dispatching strategy. All algorithms share the same snapshot-to-command interface and a common utility toolbox, making them easy to compare, extend, and benchmark under identical simulation conditions.

3.1 Unified Scheduling Framework

3.1.1 Snapshot and Command

Every scheduling round begins with a read-only snapshot captured from the data layer, containing:

- All vehicles: current positions, battery, payload, status, and on-board tasks.
- All tasks with their lifecycle states.
- All charging stations: positions, capacity, queue length, and load pressure.
- The current simulation time t_{sim}.
- Cached shortest-path queries over the road network.

The algorithm returns a list of commands. Each command specifies one vehicle's *next step*:

- **Deliver:** go to a task location (optionally loading a batch at the warehouse).
- **Charge:** go to a charging station.
- **Return:** go back to the warehouse.
- **Go to node:** move to an arbitrary coordinate (e.g., a relay meeting point).
- **Handoff:** transfer cargo to another co-located vehicle.
- **Idle:** stay put.

Multi-stop delivery is achieved by repeated snapshot-to-decision cycles: after each leg completes, the vehicle becomes idle and the scheduler is invoked again.

3.1.2 Shared Utility Toolbox

A shared utility module centralizes logic that every strategy would otherwise re-implement. Table **??** summarizes the main capability groups; developers can use these defaults as-is or override only the task-selection step:

Table 3.1: Shared Utility Capability Groups

Category	Capabilities
Query & filter	Road-network distance, payload-feasible task filtering, nearest task/station lookup
Battery & charge	Reachability checks, chain-distance estimation with mid-route recharge, low-battery detection, optimal station selection
Route ordering	Greedy nearest-neighbor chaining, MST-based visit ordering
Command construction	Standard builders for deliver, charge, return, handoff, and idle actions
Decision templates	Warehouse and en-route decision flows with built-in battery safety
Relay helpers	Path midpoint calculation, cargo-transfer feasibility checks

To implement a custom scheduler, one typically overrides only the task-picking logic; battery checks, charging fallback, route ordering, and command construction are handled automatically by the shared templates.

3.1.3 Battery Pre-Check and Mid-Route Recharge

Before dispatching a vehicle, the system verifies battery feasibility for the entire planned chain, simulating mid-route charging detours when direct travel is insufficient. Algorithm **??** outlines this chain-distance estimation procedure.

Algorithm 1 Battery Pre-check with Mid-Route Recharge

function EstimateChainDistance(vehicle, waypoints, snapshot)
    $cur \leftarrow$ vehicle position, $battery \leftarrow$ vehicle battery level
    $totalDist \leftarrow 0$, $\eta \leftarrow$ unit energy consumption, $\gamma \leftarrow$ safety margin (1.15)
    for each *target* in waypoints **do**
        loop
            $need \leftarrow$ energy to reach *target* (+ station buffer if required) $\times\gamma$
            if $battery \geq need$ **then**
                Advance *cur* to *target*; update *totalDist* and *battery*; **break**
            end if ▷ Insufficient: detour to best reachable transit station, recharge to 90%
            $station \leftarrow$ best transit station from *cur* toward *target*
            if no reachable station **then return** infeasible
            end if
            Detour to *station*; reset *battery* to charge target; continue loop
        end loop
    end for
    return *totalDist*
end function

3.1.4 Decision Templates

Most per-vehicle strategies follow a two-context pattern: warehouse decisions and en-route decisions, as shown in Algorithm **??**.

Algorithm 2 Unified Per-Vehicle Decision Template

Require: Snapshot *snap*, custom task-picking rule
Ensure: List of commands
commands ← empty list ▷ En-route vehicles: threshold charge, next undelivered task, or return
for each en-route idle vehicle v **do**
    Append en-route decision for v
end for ▷ Warehouse vehicles: pick batch via strategy-specific logic
claimed ← empty set
for each warehouse idle vehicle v **do**
    cmd ← warehouse decision for v using the custom task-picking rule
    if *cmd* assigns new tasks **then**
        claimed.update(assigned task ids)
    end if
    commands.append(*cmd*)
end for
return *commands*

At the warehouse, the template sorts the picked batch via greedy route ordering, checks full-chain battery feasibility, and issues either a deliver command (loading all tasks) or a transit charge command if the first hop requires recharging first.

3.2 Greedy Baseline Strategies

Four greedy baselines differ only in how they sort candidate tasks before greedily building a maximal feasible prefix batch. All share the same warehouse and en-route decision templates.

3.2.1 Nearest Task First

Sorts unclaimed tasks by distance from the vehicle; greedily adds tasks in nearest-first order while the chain remains payload- and battery-feasible. The simplest and fastest baseline, suitable when response time matters.

3.2.2 Heaviest Task First

Sorts by descending cargo weight (tie-break: distance). Assigns heavy cargo early to prevent payload-constrained vehicles from being stuck with only light tasks later.

3.2.3 Earliest Deadline First

Sorts by ascending deadline (tie-break: distance). A classic real-time scheduling heuristic (EDF) that prioritizes time-sensitive deliveries.

3.2.4 Priority First

Sorts by descending priority (tie-break: distance). Suitable when VIP or urgent orders must be served first regardless of distance.

All four follow the batch-building pattern shown in Algorithm **??**.

Algorithm 3 Greedy Maximal Feasible Batch (shared by all four baselines)

function PICKBATCH(vehicle, tasks, snapshot, sort rule)
  candidates ← payload-feasible tasks sorted by the given rule
  chosen ← [ ]
  for each task t in *candidates* **do**
    ordered ← greedy route order for *chosen* ∪ $\{t\}$
    if chain distance infeasible **then break**
    end if
    chosen ← *chosen* ∪ $\{t\}$
  end for
  return *chosen*
end function

3.3 Search-Based Batch Loaders

These strategies replace the greedy prefix rule with explicit search over feasible task subsets, using a unified batch chain score as the objective.

3.3.1 DFS Score Search

Enumerates all **maximal feasible batches**—subsets where no additional single task can be added without violating payload or battery constraints—via depth-first search with payload pruning. Selects the batch with the highest predicted chain score. Guarantees the optimal batch per vehicle per decision round among maximal feasible sets, at exponential worst-case cost (mitigated by small pending queues in practice).

3.3.2 SA Score Search

Uses simulated annealing to approximate the DFS optimum over the same maximal-feasible batch space. Neighbor moves add, remove, or swap tasks within the candidate pool; the cooling schedule balances exploration and exploitation. Faster than DFS when pending task count is large, with a reheat mechanism to escape local optima.

3.3.3 RL Batch

A hybrid approach combining heuristic pruning with reinforcement learning:

- **Candidate pool:** Build a nearest-first greedy prefix, then cap the pool size (default 7 tasks).
- **Batch selection:** Run DFS score search within the capped pool to pick the best maximal batch.
- **Online learning:** Maintain a Q-table over state–action pairs where state encodes nearest-count and battery tier; update Q-values from observed batch rewards after delivery completion (ϵ-greedy exploration with decay).

This design keeps per-decision latency bounded while allowing the policy to adapt across repeated simulations.

3.4 Multi-Vehicle Coordination Strategies

These strategies go beyond independent per-vehicle decisions and explicitly coordinate fleet-level task allocation.

3.4.1 Regional Planning

When pending tasks ≥ 5:

1. Cluster tasks via K-means ($k = 5$) into geographic *packages*, sorted by total weight.

2. Idle warehouse vehicles *claim* packages: a vehicle that can carry the entire package competes on predicted batch score; if no vehicle fits, defer if a returning empty vehicle could take it, otherwise assign the best-scoring feasible subset.

3. Remaining idle vehicles pick up leftover packages, prioritizing the vehicle farthest from the warehouse.

For fewer than 5 pending tasks, the strategy degrades to Nearest Task First.

3.4.2 Cluster Auction MAS

Models each idle warehouse vehicle as an independent **agent**. Tasks are clustered into lots (K-means packages or single-task lots when pending < 5). Agents submit bids:

$$\text{bid}(v, lot) = \text{predicted score}(v, lot) - \alpha \times d(v, \text{lot centroid}) \tag{3.1}$$

A central coordinator runs a **sequential auction**: lots are offered in descending weight order; the highest feasible full-package bid wins (one lot per vehicle per round). If no full-package bid exists, partial-batch bids are accepted. This multi-agent design distributes decision-making while avoiding the combinatorial explosion of a single global optimizer.

3.4.3 Relay Handoff

Enables **in-route cargo exchange** between nearby vehicles:

1. Warehouse batching follows the Nearest Task First rule.

2. Each round, scan all non-stranded vehicle pairs outside the warehouse whose road-network distance is below a proximity threshold.

3. Merge on-board tasks, K-means split into two clusters; the lighter-load vehicle takes the lighter cluster (ascending weight order) and the other takes the remainder, subject to payload feasibility.

4. Vehicles meet at the path midpoint; the first arrival waits; upon co-location, a handoff command transfers cargo.

5. Each vehicle pair may exchange at most once per outbound-return cycle to prevent oscillation.

This strategy explores cooperative delivery patterns that single-vehicle algorithms cannot achieve, such as rebalancing cargo mid-route when two vehicles converge.

3.5 Algorithm Complexity Summary

Table **??** summarizes the time complexity of all ten algorithms, assuming V idle vehicles and T pending tasks per decision round.

Table 3.2: Algorithm Complexity Comparison

Algorithm	**Time Complexity**	**Characteristics**
Nearest Task First	$O(V \cdot T \log T)$	Simplest greedy baseline
Heaviest Task First	$O(V \cdot T \log T)$	Weight-prioritized greedy
Earliest Deadline First	$O(V \cdot T \log T)$	Deadline-prioritized (EDF)
Priority First	$O(V \cdot T \log T)$	Priority-prioritized greedy
DFS Score Search	$O(2^m)$	Optimal maximal batch; m = pool size
SA Score Search	$O(N_{\text{iter}} \cdot m)$	SA approximation of DFS
RL Batch	$O(2^k + \|Q\|)$	Capped pool ($k \leq 7$) + Q-table lookup
Regional Planning	$O(k \cdot V \cdot T)$	K-means + per-package scoring
Cluster Auction MAS	$O(k \cdot V \cdot T)$	K-means + sequential auction
Relay Handoff	$O(V^2 \cdot T)$	Pair scan + handoff negotiation

3.6 Extending with Custom Algorithms

Adding a new strategy requires three steps:

1. Implement a new scheduler class that reads the snapshot and returns commands.

2. Register the class in the dynamic scheduler registry.

3. Select the new strategy name in the YAML configuration file.

For most custom heuristics, overriding only the task-picking logic inside the warehouse decision template is sufficient; the shared utilities handle battery safety, charging fallback, route ordering, and command construction automatically.

Chapter 4

Experiment Design

4.1 Experimental Environment

The experiments are conducted in the following environment:

- **Hardware:** Intel Core i7-12700H, 16GB RAM, Windows 11
- **Software:** Python 3.12, FastAPI, NetworkX, OSMnx
- **Road Network:** Real OpenStreetMap road network (Panyu District, Guangzhou), with Dijkstra shortest-path routing
- **Execution Mode:** Headless backend execution, bypassing the frontend to eliminate rendering and network overhead
- **Time Model:** Simulation time with default speed factor 120; all task timestamps, deadlines, and metrics use simulation seconds

A dedicated benchmark runner script orchestrates all experiments, ensuring every algorithm runs under the same road network, fleet parameters, and task-generation seed.

4.2 Three-Scale Configuration

To systematically evaluate the performance of each algorithm, three problem scales are designed as shown in Table **??**. Parameters not listed in the YAML files inherit system defaults.

Table 4.1: Three-Scale Experimental Configuration

Parameter	**Small**	**Medium**	**Large**
Small vehicles	2	2	4
Medium vehicles	1	3	6
Large vehicles	0	1	2
Total fleet	**3**	**6**	**12**
Charging stations	1	2	4
Charging slots (default)	2	3	4
Total task budget	**20**	**80**	**200**
Initial tasks	3	5	12
Generation interval (sim. s)	5–15	5–15	5–15
Batch size	2–4	5–10	2–4
Max pending	5	20	40
Task generation cutoff (sim. s)	3600	7200	14400

Note that the task generation cutoff controls when new task generation stops, not when the simulation ends. After the cutoff, the simulation continues until all existing tasks are completed or timed out.

4.3 Algorithm Set

Ten dynamic scheduling strategies are benchmarked:

Table 4.2: Benchmarked Algorithms

Category	**Strategy**	**Description**
Greedy baseline	Nearest Task First	Nearest-first maximal batch
Greedy baseline	Heaviest Task First	Heaviest-first maximal batch
Greedy baseline	Earliest Deadline First	Earliest-deadline-first (EDF)
Greedy baseline	Priority First	Highest-priority-first
Search-based	DFS Score Search	DFS over maximal feasible batches
Search-based	SA Score Search	Simulated annealing batch search
Search-based	RL Batch	Nearest-pool + DFS + Q-learning
Multi-vehicle	Regional Planning	K-means packages + vehicle claiming
Multi-vehicle	Cluster Auction MAS	Sequential auction over clustered lots
Multi-vehicle	Relay Handoff	In-route cargo relay between nearby vehicles

4.4 Experimental Methodology

4.4.1 Shared-Seed Protocol

To eliminate the impact of task-generation randomness on algorithm comparison, we adopt a **shared-seed protocol** rather than independent random runs per algorithm (shown in Figure **??**):

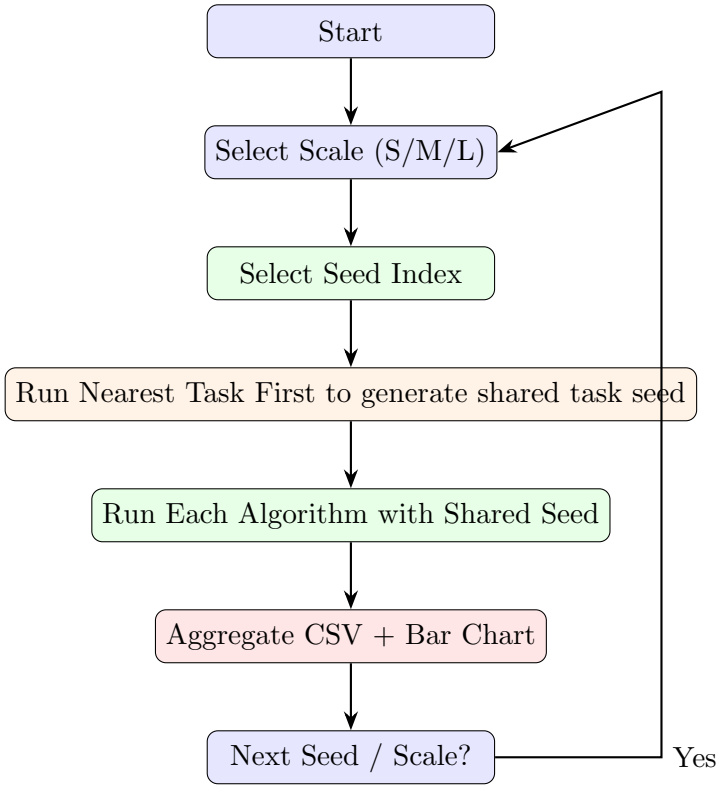


Figure 4.1: Shared-Seed Experimental Flow

Key design decisions:

- **Seed generation:** For each (scale, seed) pair, Nearest Task First runs first without a preset seed. The resulting task sequence is saved for reuse.
- **Fair comparison:** All ten algorithms replay the *identical* task sequence by loading the same seed file; only the scheduling strategy differs.
- **Seed counts:** Small scale uses 2 seeds; medium and large scales use 1 seed each, yielding 4 independent task sequences across all scales.
- **Total runs:** 4 seeds $\times$ 10 algorithms = 40 headless simulation runs.
- **Configuration isolation:** Each run writes its own configuration and log under a dedicated experiment output directory.

4.4.2 Benchmark Runner Implementation

The benchmark runner automates the following steps:

1. Load base YAML configs for small, medium, and large scales.
2. For each seed: generate (or reuse) the shared task seed via Nearest Task First.
3. Iterate all ten algorithms, changing only the strategy and the seed file reference.
4. Invoke the headless backend for each run.
5. Parse the results block from each experiment log.
6. Write per-seed CSV summaries and comparison bar charts.

This pipeline ensures reproducibility: re-running the benchmark can skip already-completed runs and resume from incomplete seeds.

4.5 Evaluation Metrics System

Each simulation run records the following metrics (all time-based values in simulation seconds unless noted):

Table 4.3: Evaluation Metrics System

Metric	**Calculation**	**Interpretation**
Completion Rate	$\frac{N_{\text{completed}}}{N_{\text{total}}}$	Task processing capability
On-time Rate	$\frac{N_{\text{on-time}}}{N_{\text{completed}}}$	Timeliness
Total Score	$\sum \text{score}(T_i)$	Overall reward
Avg Completion Time	$\frac{1}{N}\sum(t_{\text{finish}} - t_{\text{create}})$	Response speed (sim. s)
Throughput	$\frac{N_{\text{completed}}}{\text{sim. hours}}$	System efficiency
Total Distance	$\sum_v \text{dist}(v)$	Energy cost
Distance StdDev	$\sigma(\{\text{dist}(v)\})$	Load balance
Simulation Duration	—	Total simulation time consumed
Wall Duration	—	Real-world runtime (reference only)
Stranded Vehicles	—	Safety / robustness indicator

4.5.1 Algorithm Selection Criteria

When comparing algorithms, we use a multi-criteria decision approach:

- **Primary criterion:** Total Score — directly reflects the final payoff of the strategy under the unified scoring formula.
- **Secondary criteria:** Completion Rate and On-time Rate — reflect service quality.

- **Auxiliary criteria:** Avg Completion Time and Throughput — reflect efficiency in simulation time.
- **Reference criteria:** Total Distance, Distance StdDev, and Stranded Vehicles — reflect resource utilization and robustness.
- **Runtime reference:** Wall-clock duration is logged but not used for ranking, since all algorithms share the same simulation-time model.

Per-seed results are aggregated into CSV files and visualized as grouped bar charts (Total Score and Completion Rate) for direct cross-algorithm comparison within each scale.

Chapter 5

Results and Analysis

This chapter presents the dynamic scheduling benchmark results from `experiments/dynamic_ben`. All algorithms under the same scale share an identical task-generation seed; small scale is averaged over two seeds, while medium and large scales use one seed each. All time-based metrics are in simulation seconds.

5.1 Small-Scale Results

5.1.1 Performance Table

The small-scale configuration uses 3 vehicles, 1 charging station, and a task budget of 20. Table **??** reports the mean over seed 1 and seed 2 (11–12 tasks settled per run).

Table 5.1: Small-Scale Dynamic Benchmark Results (mean of 2 seeds)

Algorithm	**Comp.(%)**	**On-time(%)**	**Total Score**	**Avg Time (s)**
RL Batch	100.0	73.3	**1262.4**	5974
SA Score Search	100.0	72.5	1243.9	4190
DFS Score Search	100.0	**81.7**	1242.4	4888
Nearest Task First	100.0	68.3	1238.4	5177
Regional Planning	100.0	72.5	1171.0	5304
Priority First	100.0	72.5	1166.4	4564
Heaviest Task First	100.0	63.3	1066.1	5531
Cluster Auction MAS	100.0	64.2	1037.4	5155
Relay Handoff	100.0	64.2	1030.6	5793
Earliest Deadline First	100.0	72.5	940.7	4717

chapters/comparison0.png

Figure 5.1: Small-scale total score comparison seed 1.

Figure 5.2: Small-scale total score comparison seed 2.

5.1.2 Analysis

- All ten algorithms achieve **100% completion** on both seeds; differentiation comes from total score and on-time rate.
- Search-based batch loaders (**RL Batch**, **SA Score Search**, **DFS Score Search**) occupy the top tier (1242–1262 mean score). **DFS Score Search** also yields the highest mean on-time rate (81.7%).
- **Earliest Deadline First** scores lowest (940.7 mean), indicating that deadline-only sorting is insufficient when batch loading and battery constraints dominate.
- Seed 2 is more volatile: **Earliest Deadline First** drops to 699.1 while **DFS Score Search** reaches 1142.6 with 83.3% on-time, confirming sensitivity to task sequence even under shared infrastructure.
- Cooperative strategies (**Relay Handoff**, **Cluster Auction MAS**) do not outperform greedy baselines at this scale, likely because the fleet is small and

handoff overhead is not yet justified.

5.2 Medium-Scale Results

5.2.1 Performance Table

The medium-scale run uses 6 vehicles, 2 charging stations, and settles 49 tasks. Results from seed 1 are shown in Table **??**.

Table 5.2: Medium-Scale Dynamic Benchmark Results (seed 1, 49 tasks)

Algorithm	**Comp.(%)**	**On-time(%)**	**Total Score**	**Avg Time (s)**
DFS Score Search	100.0	53.1	**3824.3**	6990
RL Batch	100.0	53.1	3795.3	6311
SA Score Search	100.0	51.0	3716.2	7353
Nearest Task First	100.0	**57.1**	3377.2	7118
Relay Handoff	100.0	51.0	2913.1	7503
Priority First	100.0	40.8	2824.1	8869
Regional Planning	100.0	49.0	2385.3	7099
Cluster Auction MAS	100.0	49.0	2385.3	7145
Heaviest Task First	100.0	40.8	2112.6	9240
Earliest Deadline First	100.0	46.9	1716.3	8190

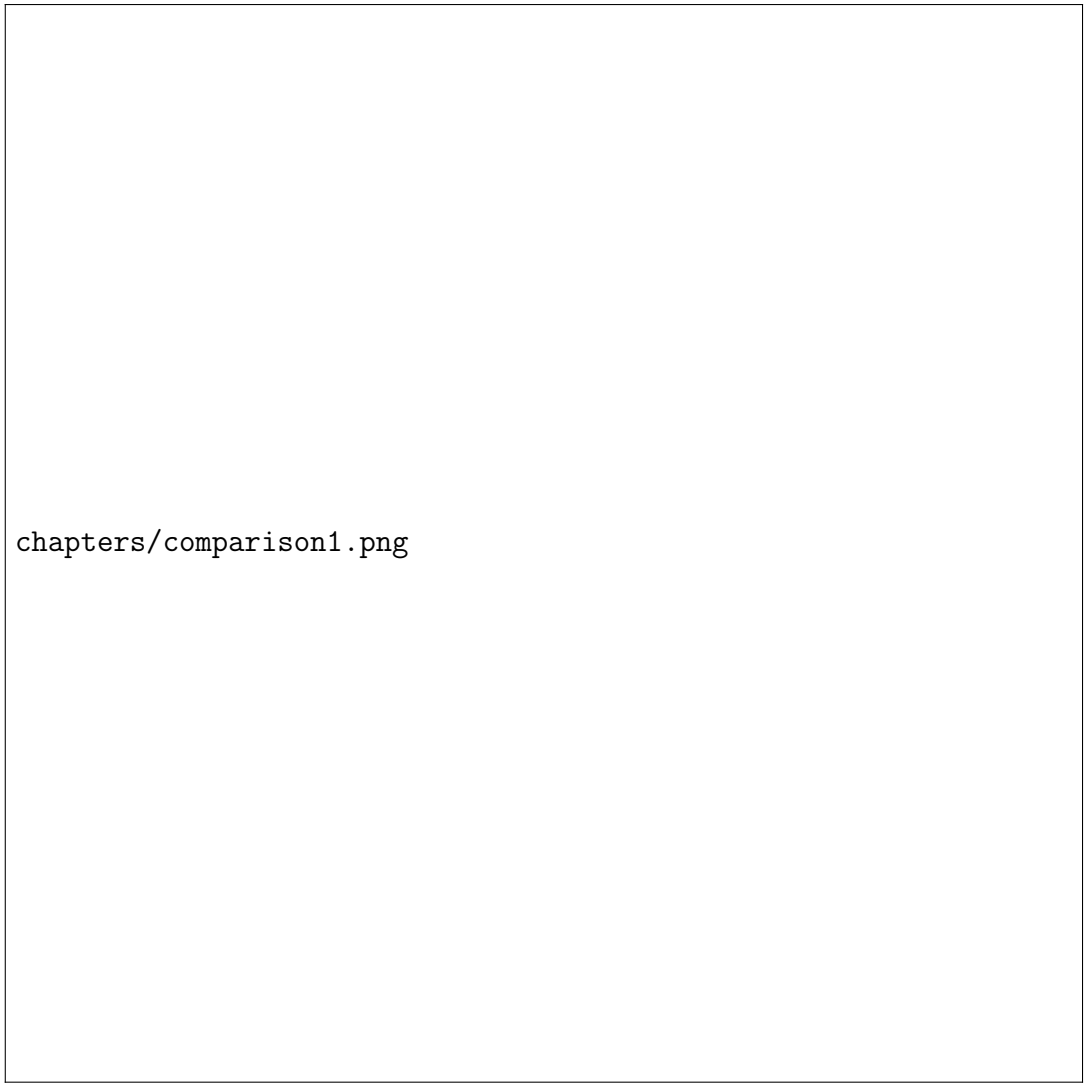


Figure 5.3: Medium-scale total score comparison (seed 1, 49 tasks).

5.2.2 Analysis

- Completion remains **100%** for all algorithms, but on-time rates fall to 41–57%, reflecting tighter deadlines relative to fleet capacity.
- **DFS Score Search** (3824.3) and **RL Batch** (3795.3) clearly outperform greedy baselines, validating explicit batch-score optimization under higher load.
- Simple greedy rules diverge: **Nearest Task First** stays competitive (3377.2) while **Earliest Deadline First** collapses to 1716.3 — deadline sorting alone cannot compensate for poor batch composition.
- Multi-vehicle coordinators (**Regional Planning**, **Cluster Auction MAS**) score 2385.3 with identical outcomes on this seed, suggesting similar package-decomposition behavior but no advantage over search-based per-vehicle batching here.

- The score gap between best and worst widens to 2108 points (3824.3 vs. 1716.3), showing that algorithm choice matters substantially at medium scale.

5.3 Large-Scale Results

5.3.1 Performance Table

The large-scale run uses 12 vehicles, 4 charging stations, and settles 32 tasks within the generation window. Table **??** lists seed 1 results.

Table 5.3: Large-Scale Dynamic Benchmark Results (seed 1, 32 tasks)

Algorithm	**Comp.(%)**	**On-time(%)**	**Total Score**	**Avg Time (s)**
Cluster Auction MAS	96.9	96.8	**3278.7**	1194
SA Score Search	100.0	**96.9**	3221.3	1089
Regional Planning	100.0	96.9	3045.0	1530
Heaviest Task First	100.0	93.8	2982.8	1395
RL Batch	100.0	93.8	2907.9	1305
Earliest Deadline First	100.0	93.8	2907.9	1323
DFS Score Search	100.0	93.8	2907.9	1305
Priority First	100.0	93.8	2727.1	1377
Nearest Task First	100.0	96.9	2652.6	1368
Relay Handoff	100.0	93.8	2244.0	1764

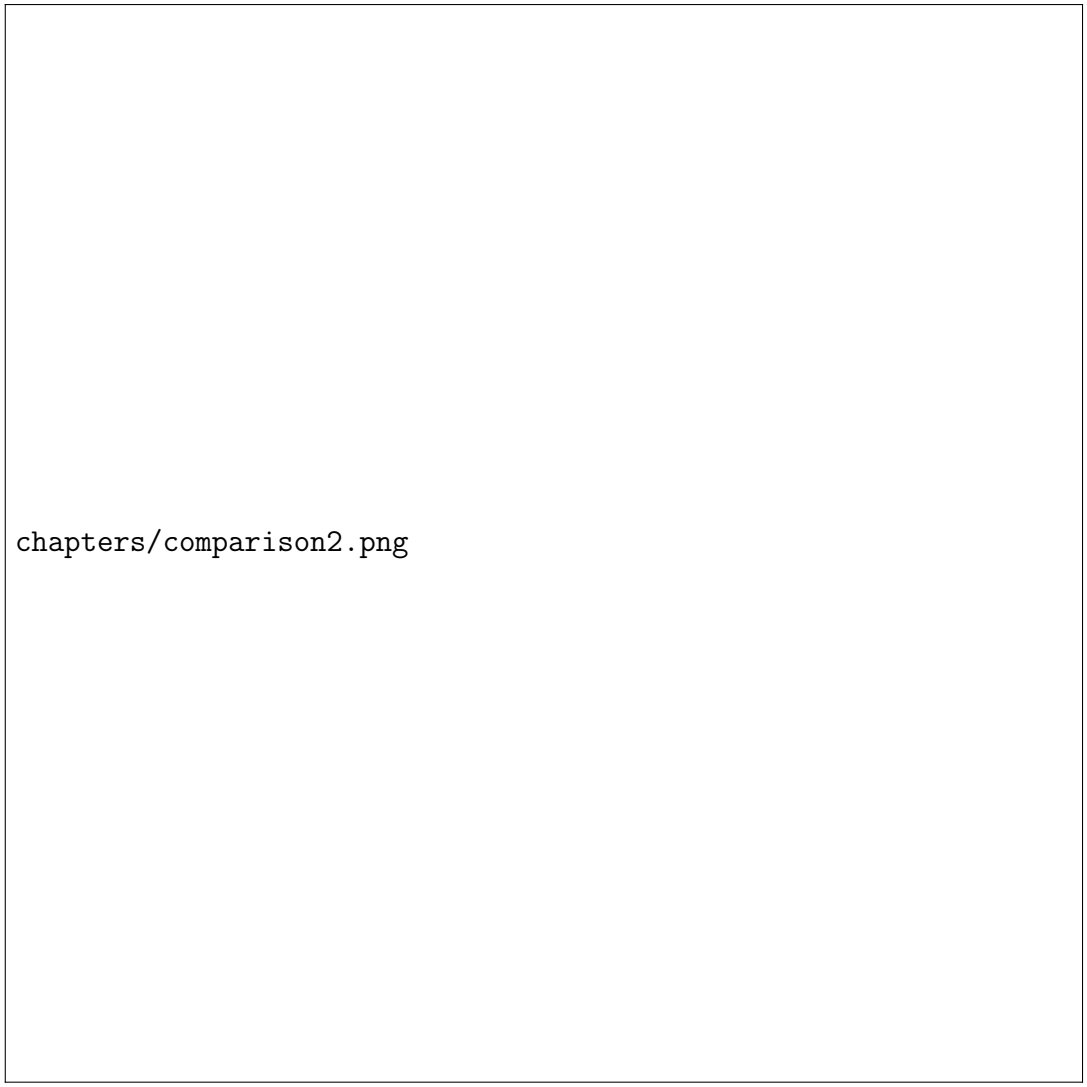


Figure 5.4: Large-scale total score comparison (seed 1, 32 tasks).

5.3.2 Analysis

- Most algorithms complete all 32 tasks; **Cluster Auction MAS** stops at 31/32 (96.9% completion) with one timeout, yet still achieves the highest total score (3278.7) on this seed.
- **SA Score Search** balances score (3221.3) with full completion and the joint-best on-time rate (96.9%).
- **Relay Handoff** scores lowest (2244.0) despite 100% completion — relay negotiation adds route overhead without sufficient cargo-rebalancing benefit on this instance.
- Several batch-search variants (**RL Batch**, **DFS Score Search**, **Earliest Deadline First**) converge to identical scores (2907.9), indicating diminishing returns of search when the pending pool is small at each decision step.
- Compared with medium scale, absolute scores drop because fewer tasks are

settled before the generation cutoff, highlighting that scale configuration affects both difficulty and score magnitude.

5.4 Comparison with Static Gurobi Solver

Figure 5.5: static result for gurobi.

As an supplementary baseline, we also ran the static omniscient solver (Gurobi) on the same task-generation seeds. Table **??** summarizes those results.

Table 5.4: Static Omniscient Solver Results (Gurobi)

Scale	**Total Score**	**Completed / Total**	**Completion Rate**
Small	877.9	10 / 10	100.0%
Medium	3003.6	20 / 20	100.0%
Large	4407.0	38 / 39	97.4%

The static solver results differ substantially from the dynamic benchmark and do not consistently dominate:

- At **small** and **medium** scales, the best dynamic strategies (**RL Batch** at 1262.4; **DFS Score Search** at 3824.3) *exceed* the static scores (877.9 and 3003.6 respectively).
- At **large** scale, the static total (4407.0) appears higher in absolute terms, but it covers a different settled task count (38/39 vs. 32/32 for most dynamic runs), so direct comparison is limited.
- We suspect the gap is partly caused by **time-penalty modeling** in the MIP formulation: overdue and timing-related penalty terms may push the solver toward conservative routes that score poorly under our unified dynamic scoring function, and in some cases yield solutions worse than online heuristics.

5.5 Cross-Scale Summary

Table **??** highlights the best-performing dynamic strategy per scale.

Table 5.5: Best Dynamic Strategy per Scale

Scale	Best Algorithm	Score	On-time	Notes
Small	RL Batch	1262.4	73.3%	Mean of 2 seeds; DFS best on-time (81.7%)
Medium	DFS Score Search	3824.3	53.1%	Search-based batching clearly wins
Large	Cluster Auction MAS	3278.7	96.8%	One timeout; SA Score Search best balanced

Key findings across all experiments:

- **No single algorithm wins everywhere.** Search-based loaders excel at medium scale; auction-based coordination peaks at large scale; RL/SA variants are strong at small scale.
- **Completion vs. quality.** All dynamic runs achieve near-perfect completion; on-time rate and total score are the primary discriminators, especially at medium scale where on-time drops below 60%.
- **Greedy baselines remain useful. Nearest Task First** stays in the upper tier at small and medium scales with minimal implementation cost.
- **Cooperative strategies are scale-dependent. Relay Handoff** underperforms at all scales tested; **Cluster Auction MAS** becomes competitive only when the fleet and task pool are large enough.
- **Static vs. dynamic.** Gurobi static planning does not reliably beat online dynamic policies under the same scoring rules, suggesting that model fidelity (especially time penalties) matters as much as optimality guarantees.

Chapter 6

Conclusion and Reflections

6.1 Project Summary

This project designed and implemented NEFT, a simulation platform for electric logistics fleet cooperative dispatching, and evaluated ten dynamic scheduling strategies under controlled, reproducible conditions. The main outcomes are:

6.1.1 Simulation System

- Built a graph-based road network (OpenStreetMap / OSMnx) with Dijkstra shortest-path routing and a simulation-time clock decoupled from wall-clock execution.
- Modeled three vehicle types, dynamic task generation with shared seeds, charging-station queuing, and a unified scoring function used for both settlement and batch-selection objectives.
- Delivered a snapshot-to-command scheduling interface with shared utility modules, enabling fair comparison and rapid prototyping of new strategies.
- Implemented a FastAPI + WebSocket + AMap visualization front-end for real-time monitoring.

6.1.2 Algorithm Portfolio

- **Greedy baselines (4):** Nearest Task First, Heaviest Task First, Earliest Deadline First, Priority First.
- **Search-based batch loaders (3):** DFS Score Search, SA Score Search, RL Batch.
- **Multi-vehicle coordination (3):** Regional Planning, Cluster Auction MAS, Relay Handoff.
- All strategies share battery pre-check, mid-route recharge simulation, and charging-station load balancing through common decision templates.

6.1.3 Experimental Findings

- Ran 40 headless dynamic benchmarks (4 shared seeds $\times$ 10 algorithms) under identical task sequences per seed.
- Dynamic algorithms achieve near-perfect completion; total score and on-time rate separate strategies, especially at medium scale (on-time 41–57%).

- **DFS Score Search** and **RL Batch** lead at medium scale (≈3800 points); **RL Batch** and search variants lead at small scale; **Cluster Auction MAS** scores highest at large scale on the tested seed.
- A supplementary Gurobi static solver baseline did not consistently outperform dynamic policies; at small and medium scales several dynamic strategies scored higher, possibly because time-penalty terms in the MIP model distort the objective relative to the online scoring function.

6.2 Lessons Learned

1. **Fair comparison requires infrastructure, not just algorithms.** The snapshot interface and shared task seeds were essential; without them, score differences would reflect random task noise rather than strategy quality.
2. **Batch loading dominates medium-scale performance.** When pending queues grow, explicit search over feasible batches (DFS / RL) substantially beats single-key greedy sorting.
3. **Cooperation has a scale threshold.** Relay and auction mechanisms need sufficient fleet size and geographic task spread; otherwise coordination overhead hurts more than it helps.
4. **Static optima ≠ dynamic benchmark winners.** Exact solvers are only meaningful when the mathematical model faithfully encodes the same penalties and timing semantics as the simulator; mismatched time-penalty modeling can make "optimal" static plans score worse than simple online heuristics.
5. **Visualization accelerates debugging.** Map-based route display exposed charging and routing edge cases that log inspection alone would have missed.

6.3 Future Work

- Refine the static MIP formulation so that deadline and overdue penalties align exactly with the dynamic scoring module, enabling a fair static-vs-dynamic gap analysis.
- Extend benchmarks with more seeds per scale and report mean ± std. dev. for statistical confidence.
- Tune cooperative strategies (relay thresholds, auction bid weights) and test on denser task streams where handoff should pay off.
- Explore hybrid policies that switch between greedy, search, and multi-agent modes based on pending-queue size and fleet utilization.

6.4 Team Reflections

This project took us from textbook scheduling concepts to a working system where vehicles move on a real map, charge at stations, and compete under a unified score. Implementing ten strategies on one interface showed how much engineering effort lies beyond the core algorithm — battery safety, seed reproducibility, and simulation-time semantics all shaped the final rankings as much as the dispatching logic itself.

The experimental phase changed how we evaluate algorithms: a single impressive run is misleading, but shared seeds across ten strategies revealed clear patterns — search wins at medium load, coordination matters at large scale, and simple nearest-neighbor rules remain a strong baseline. Working in a team mirrored real software practice: simulation core, algorithms, visualization, and analysis had to integrate through clear contracts, much like the snapshot-to-command design we built for the schedulers themselves.